

API & Web Security Risk Analysis Report

Prepared for	Future Interns — Cyber Security Track (Task 3)
Repository	FUTURE_CS_03
Assessment Type	Passive API / HTTP Security Configuration Review
Tools Used	Postman (Collection Runner + Tests), Browser DevTools
Scope	Publicly accessible HTTP response headers, cookies, and API responses
Test Coverage	39 automated assertions across Transport, Headers, Cookies, Data Exposure, XSS, SQLi, Injection, Con
Result Summary	35 / 39 checks passed (89.7%) — 4 checks failed

Disclosure note: This assessment was limited strictly to passive, non-intrusive checks — inspection of HTTP response headers, cookie attributes, and response content already returned by the server during normal browsing. No authentication bypass, exploitation, fuzzing, or intrusive testing was performed against the target.

1. Executive Summary

An automated security configuration review was performed against a live e-commerce web/API endpoint using a 39-point Postman test suite covering transport security, HTTP security headers, cookie attributes, sensitive data exposure, and common injection patterns (XSS, SQLi, open redirects, path traversal). The site passed **35 of 39 checks (89.7%)**, indicating a generally mature security baseline. Core protections — HTTPS enforcement, HSTS, secure/HttpOnly/SameSite cookies, absence of stack traces, database errors, and payment data in responses — are all correctly implemented.

However, **three confirmed gaps in HTTP security headers** were identified that reduce the site's defense-in-depth against client-side attacks such as cross-site scripting (XSS) and unwanted data leakage via the Referrer header. A fourth finding (possible internal admin email exposure) returned an inconclusive automated result and requires manual verification before being treated as confirmed.

Severity	Count	Findings
Medium	1	Missing enforced Content-Security-Policy (CSP)
Low	2	Missing Referrer-Policy · Missing Permissions-Policy
Informational	2	Debug header exposed in production · Internal server identifier exposed
Needs Verification	1	Possible admin email pattern in response (automated false-positive risk)

Name	Value	Domain	Path	Expires	HttpOnly
__cf_bm	m7BHx13Q4Tw0o2rfmUJ20NqXJNHySvdRfgAg1QL...	dyson.in	/	Fri, 03 Jul 2026 06:42:26 GMT	true
PHPSESSID	91752c486e379575c33fa7757777d21	www.dyson.in	/	Fri, 03 Jul 2026 07:12:37 GMT	true
X-Magento-Vary	05cbee7556b6aaf3b10537bc2be1d26c40694501f3f...	www.dyson.in	/	Fri, 03 Jul 2026 07:12:37 GMT	true

key	200
status	PX, 63 Jul 2028 06:17:05 GMT
date	Tue, 03 Jul 2028 06:17:05 GMT
content-type	text/html; charset=UTF-8
cf-ray	wf53ucba09f9m7-CJH
cf-cache-status	DYNAMIC
age	1260
cache-control	No-store, no-cache, must-revalidate, max-age=0
expires	Fri, 03 Jul 2028 06:58:10 GMT
server	cloudflare
strict-transport-security	max-age=31536000; includeSubDomains; preload
x-y	Access-Crossing-Cookie
pragma	cache
content-security-policy-report-only	font-src * fonts.gstatic.com as * gip.gcp.com * gstatic.com * cloudfront.net * checkout.com date.*self unsafe-inline frame-ancestors 'self' frame-src *.brightcove.net *.cloudfront.net * assets.brightcove.com
v-cache	MSS, VET, HT
v-cache-hits	0, 0, 0
v-varied-by	cache-dbd4374-DUB, cache-dbd4374-DUB, cache-dbd4374-DUB, cache-vry0532-MRS
report-to	[{"group":"report-endpoint","max_age":1088640,"endpoints":["url":"https://us-east-1-lambda.amazonaws.com/_aws/reportOnly/JS"]}]
incorporation	00-WheelP85bcVndGw4e4E1u378W5SR+27Uz7d5dMStu42-01
v-cache-page	1
v-content-type-options	nosniff
v-cache-key	qy3ZtYpRr1jpwUo++
v-frame-options	SAMEORIGIN
v-platform-server	L-QvF8B5kaC2358MF1_L-QvF8B5kaC2358MF3
v-dnsip	617630566176.967027V56/V5774
v-exp-protection	1, mode=click
content-encoding	gzip
alt-svc	h3="AAEP", max=88400

2. Methodology

The assessment followed a passive reconnaissance and response-inspection approach, consistent with a non-intrusive API/web security review:

- 1 Captured live HTTP response headers returned by the target's CDN/edge (Cloudflare) and origin application.
- 2 Inspected cookies set during a normal browsing session for Secure, HttpOnly, and SameSite attributes.
- 3 Ran a 39-assertion Postman test collection covering transport security, header hardening, cookie security, sensitive data exposure, XSS/SQLi/injection reflection, content handling, and CSRF protection.
- 4 Reviewed raw response bodies and headers for accidental disclosure of internal infrastructure details, debug information, or sensitive data.
- 5 Classified each failed or borderline check by business impact and assigned a risk rating (Critical / High / Medium / Low / Informational).

Tools: Postman (Collection Runner with automated test assertions), Browser DevTools (Network / Application tabs for headers and cookies).

Scope boundary: No credentials were tested, no requests were sent that could modify data, and no rate-limit, fuzzing, or exploitation attempts were made. All findings below are derived purely from server responses returned during ordinary, authorized page loads.

3. Automated Test Results Overview

35 of 39 assertions passed. Categories below reflect the Postman test suite output.

Category	Passed	Failed	Notes
Transport	4/4	0	HTTPS enforced; no 4xx/5xx; acceptable response time
Headers	6/9	3	HSTS, X-Content-Type-Options, X-Frame-Options OK; CSP, Referrer-Policy, Permissions-
Cookies	4/4	0	Secure, HttpOnly, SameSite all correctly set
Data Exposure	5/6	1	No PAN/CVV/passwords/stack traces/DB errors; 1 inconclusive admin-email check
XSS	3/3	0	No reflected payloads, no raw script tags, no DOM sinks
SQLi	2/2	0	No reflected payloads or SQL error messages
Injection (Redirect/Path)	2/2	0	No open redirect or path traversal patterns
Content Handling	4/4	0	Correct Content-Type; cache headers prevent caching of cart page
CSRF	1/1	0	CSRF token present in form/meta tag

```
PASSED [HEADERS] CSP does not use unsafe-inline without nonce or hash
FAILED [HEADERS] Referrer-Policy is set | AssertionError: Referrer-Policy missing - cart URLs may leak to third parties: expected null not to be null
PASSED [HEADERS] Referrer-Policy is not 'unsafe-url'
FAILED [HEADERS] Permissions-Policy or Feature-Policy is set | AssertionError: Permissions-Policy missing - browser features not restricted: expected null not to be null
PASSED [HEADERS] Server header does not expose version info
PASSED [HEADERS] X-Powered-By header is absent
PASSED [HEADERS] X-AspNet-Version header is absent
PASSED [COOKIES] Session-like cookies have Secure flag
PASSED [COOKIES] Session-like cookies have HttpOnly flag
PASSED [COOKIES] SameSite attribute is set on all cookies
PASSED [COOKIES] Cookies with SameSite=None must have Secure flag
PASSED [DATA] No raw credit/debit card numbers (PAN) in response
PASSED [DATA] No CVV/CVC patterns in response
PASSED [DATA] No plaintext passwords or API secrets in response
PASSED [DATA] No private or internal IP addresses in response
PASSED [DATA] No stack traces or server error dumps in response
PASSED [DATA] No database errors or connection strings in response
FAILED [DATA] No internal admin email addresses exposed | AssertionError: expected '<doctype html><html lang=en-IN><h..> not to match /([a-z0-9_%+!-])+(@dyson.com|dyson\.in|internal.dyson)/i
PASSED [XSS] Response does not reflect XSS payloads unescaped
PASSED [XSS] No raw script-tag alert patterns in response
PASSED [XSS] No DOM sink patterns in inline scripts
PASSED [SQLi] Response does not reflect SQL injection payloads
```

4. Detailed Findings

F1. Content-Security-Policy (CSP) Not Enforced

MEDIUM

Description: The site currently ships only a **Content-Security-Policy-Report-Only** header rather than an enforced Content-Security-Policy. A report-only policy logs violations but does not block malicious scripts, so it provides no active protection against cross-site scripting (XSS) or unauthorized resource loading.

Evidence: Response headers show `content-security-policy-report-only` present with an allow-list (payment/analytics domains), but no corresponding enforcing `content-security-policy` header was returned.

Business Impact: If an XSS vulnerability is ever introduced elsewhere in the application, there is currently no browser-level safety net to stop injected scripts from executing, exfiltrating session data, or manipulating the checkout flow.

Recommended Fix: Move the existing report-only policy to an enforcing Content-Security-Policy header once violation reports confirm the allow-list is complete and stable in staging/production.

F2. Referrer-Policy Header Missing

LOW

Description: No Referrer-Policy header was returned. Without this header, browsers may default to sending the full referring URL — including query parameters — to third-party domains when a user navigates away from the page.

Evidence: Referrer-Policy was absent across all captured responses, including cart/checkout pages.

Business Impact: Cart or account URLs that embed session tokens, product IDs, or other parameters in the query string could be leaked to third-party analytics, advertising, or payment partner domains via the Referer header.

Recommended Fix: Add Referrer-Policy: `strict-origin-when-cross-origin` (or a stricter value such as `no-referrer-when-downgrade`) at the CDN or application layer.

F3. Permissions-Policy Header Missing

LOW

Description: No Permissions-Policy (or legacy Feature-Policy) header was found, meaning browser features such as camera, microphone, geolocation, and payment APIs are not explicitly restricted for this origin or embedded third-party content.

Evidence: Header absent in all responses reviewed.

Business Impact: Any third-party script or iframe loaded on the page (e.g. from an ad, chat widget, or payment integration) could request access to sensitive browser APIs that the site itself never uses, widening the attack surface unnecessarily.

Recommended Fix: Add a restrictive Permissions-Policy header, explicitly disabling unused features, e.g. `camera=(), microphone=(), geolocation=()`.

F4. Debug / Internal Infrastructure Headers Exposed

INFORMATIONAL

Description: Two non-standard headers were observed in production responses: `x-debug-info` (a base64-encoded value) and `x-platform-server` (an internal server/instance identifier).

Evidence: Both headers were present on the live response alongside standard Cloudflare/CDN headers.

Business Impact: While not directly exploitable on their own, debug and internal-hostname headers give an attacker reconnaissance information about backend infrastructure and may indicate that a debug/staging configuration flag is still active in production.

Recommended Fix: Strip debug and internal-routing headers at the CDN/edge before responses reach the client, and confirm debug modes are disabled in the production environment.

F5. Possible Internal Admin Email Pattern (Unconfirmed)

NEEDS
VERIFICATION

Description: One automated assertion intended to detect internal admin email addresses in the response failed. However, the failure message shows the test's expected-pattern regex itself, rather than a clearly matched email address — a strong indicator of a test-tooling artifact or overly broad regex rather than a genuine leak.

Evidence: The Postman assertion error output was inconclusive and did not display a specific matched email string.

Business Impact: If a genuine admin/internal email were exposed, it could be used for targeted phishing or account enumeration; however, this cannot be confirmed from the automated result alone.

Recommended Fix: Manually review the raw response body for this endpoint to confirm whether an internal email address is actually present before treating this as a verified finding. If confirmed, remove the address from public-facing responses and route contact requests through a generic alias instead.

```
PASSED [TRANSPORT] Response is served over HTTPS
PASSED [TRANSPORT] HTTP status is not a server error (not 5xx)
PASSED [TRANSPORT] HTTP status is not a client error (not 4xx)
PASSED [TRANSPORT] Response time is within acceptable limit (< 10s)
PASSED [HEADERS] Strict-Transport-Security (HSTS) is present
PASSED [HEADERS] HSTS max-age is at least 6 months (15768000s)
PASSED [HEADERS] X-Content-Type-Options is 'nosniff'
PASSED [HEADERS] X-Frame-Options prevents clickjacking
FAILED [HEADERS] Content-Security-Policy (CSP) is present | AssertionError: CSP header missing - no XSS mitigation layer: expected null not to be null
PASSED [HEADERS] CSP does not use unsafe-inline without nonce or hash
FAILED [HEADERS] Referrer-Policy is set | AssertionError: Referrer-Policy missing - cart URLs may leak to third parties: expected null not to be null
PASSED [HEADERS] Referrer-Policy is not 'unsafe-url'
FAILED [HEADERS] Permissions-Policy or Feature-Policy is set | AssertionError: Permissions-Policy missing - browser features not restricted: expected null not to be null
PASSED [HEADERS] Server header does not expose version info
PASSED [HEADERS] X-Powered-By header is absent
PASSED [HEADERS] X-AspNet-Version header is absent
PASSED [COOKIES] Session-like cookies have Secure flag
PASSED [COOKIES] Session-like cookies have HttpOnly flag
PASSED [COOKIES] SameSite attribute is set on all cookies
PASSED [COOKIES] Cookies with SameSite=None must have Secure flag
PASSED [DATA] No raw credit/debit card numbers (PAN) in response
PASSED [DATA] No CVV/CVC patterns in response
PASSED [DATA] No plaintext passwords or API secrets in response
PASSED [DATA] No private or internal IP addresses in response
PASSED [DATA] No stack traces or server error dumps in response
PASSED [DATA] No database errors or connection strings in response
FAILED [DATA] No internal admin email addresses exposed | AssertionError: expected ' <!doctype html> <html lang="en-IN"> <h... ' not to match /(a-z0-9_%+)-+@(dyson\.com|dyson\.in|internal\.dyson)/i
PASSED [XSS] Response does not reflect XSS payloads unescaped
PASSED [XSS] No raw script-tag alert patterns in response
PASSED [XSS] No DOM sink patterns in inline scripts
PASSED [SQLI] Response does not reflect SQL injection payloads
PASSED [SQLI] No SQL error messages in response
PASSED [INJECTION] No open redirect patterns in response links
PASSED [INJECTION] No path traversal patterns in response
PASSED [CONTENT] Content-Type header is present
PASSED [CONTENT] Response Content-Type is HTML
PASSED [CONTENT] Cache-Control prevents public caching of cart page
PASSED [CONTENT] Pragma no-cache is set (legacy proxy control)
PASSED [CSRF] Response contains a CSRF token in form or meta tag
```

5. Strengths Observed

- HTTPS is enforced site-wide with a strong HSTS policy (max-age ≥ 6 months, includeSubDomains, preload).
- All session-related cookies (PHPSESSID, X-Magento-Vary, __cf_bm) correctly set the Secure and HttpOnly flags and a SameSite attribute, protecting against cookie theft over insecure channels and basic CSRF vectors.
- X-Content-Type-Options: nosniff and X-Frame-Options: SAMEORIGIN are both present, mitigating MIME-sniffing attacks and clickjacking.
- No sensitive data — card numbers, CVV, plaintext passwords, API keys, internal IPs, stack traces, or database errors — was observed in any response.
- No reflected XSS, DOM-sink, SQL injection, open-redirect, or path-traversal patterns were detected in the tested responses.
- Cache-Control and Pragma headers correctly prevent public caching of the cart page, and a CSRF token is present in forms.
- Server and technology-revealing headers (X-Powered-By, X-AspNet-Version, detailed Server banner) are properly suppressed.

6. Prioritized Remediation Plan

Priority	Action	Owner Area
1	Promote Content-Security-Policy from report-only to enforced once allow-list is validated	App / Edge (CDN)
2	Add Referrer-Policy: strict-origin-when-cross-origin	App / Edge (CDN)
3	Add restrictive Permissions-Policy header	App / Edge (CDN)
4	Strip x-debug-info and x-platform-server headers from production responses	Platform / DevOps
5	Manually verify and, if needed, redact the possible admin-email pattern	App team

7. Conclusion

The assessed application demonstrates a solid baseline security posture, passing the large majority (89.7%) of automated checks across transport security, cookie handling, and injection resistance. The remaining gaps are concentrated in browser-enforced HTTP security headers (CSP, Referrer-Policy, Permissions-Policy) rather than exploitable vulnerabilities, and can be remediated with configuration changes at the CDN/edge layer with minimal engineering effort. Addressing the prioritized items above would move the site from a good to a strong security header posture.